

SDV_Replica_POC

Daily Work Log

Author: Chittaranjan Baral (@SJLNS)

Date: 2026-08-08

Project: SDV_Replica_POC

Log version: v1.0

1. SESSION OVERVIEW

Focus of this session: publish the SDV_Replica_POC repository scaffold to a personal, open-source GitHub account separate from company git tooling; add author/version/date metadata across the repo; build and verify a structured firmware build wrapper (**buildenv.sh**) with staged artifacts and loud failure reporting; install and verify the Arm GNU Toolchain on the local laptop; and resolve several Windows/Git-specific issues encountered along the way.

This log is written so a fresher joining the project later can follow exactly what was done, what broke, why it broke, and how it was fixed - not just the commands that were run. Yellow boxes throughout explain each issue in plain language. A glossary of terms used is at the end of this document.

2. GITHUB REPOSITORY SETUP

Step	Result
Created personal GitHub account (username: SJLNS)	OK
Confirmed separation from company git tooling (repo-local git config vs. global)	OK
Verified git installed but not on cmd PATH -> used Git Bash instead	INFO
Created empty repo on github.com/new (no auto README/license/gitignore)	OK
git remote add origin https://github.com/SJLNS/SDV_Replica_POC.git	OK
First push attempt failed: repository not found (repo not yet created on GitHub)	FAIL
Repo created on GitHub, retried push -> succeeded (192 objects)	OK
Pushed tag v0.1.0	OK

KEY ISSUE - Repository not found

git remote add only stores a URL locally; it does not create anything on GitHub's server. Root cause was pushing before creating the empty repo on GitHub's website. Fixed by creating the repo first, then retrying the push.

In plain terms: Think of 'git remote add' as writing an address on an envelope - it does not create the house at that address. The GitHub repository ('the house') has to be created separately, once, on github.com, before git can deliver anything to it. This is a one-time setup step, never needed again for this repo.

3. ARM GNU TOOLCHAIN INSTALLATION

Step	Result
Attempted download via developer.arm.com (deprecated, redirects to GitLab)	FAIL
Located gitlab.arm.com/tooling/gnu-toolchains-for-arm	INFO
GitLab Releases page empty for this project (no releases listed there)	FAIL
Switched approach: used winget instead of manual GitLab download hunting	OK
winget install Arm.GnuArmEmbeddedToolchain (v14.2.Rel1, 197 MB)	OK
Backed up user PATH before install (known winget PATH-overwrite bug on this package)	OK
Verified PATH intact after install - no data loss	OK
arm-none-eabi-gcc --version confirmed working in a fresh Git Bash window	OK

RISK NOTED - winget PATH overwrite bug

Arm.GnuArmEmbeddedToolchain has a documented history of replacing the entire user PATH variable instead of appending to it. Mitigated by exporting PATH to a backup file immediately before installing. Verified after install that all prior PATH entries (Python, VS Code, Vector tools, CTOOLS) were still present.

In plain terms: PATH is the list of folders Windows searches through whenever you type a command name (like 'git' or 'python') without giving its full location. If an installer wipes that list instead of adding to it, every other tool that relied on being 'found' via PATH would silently stop working - not because it was uninstalled, but because Windows no longer knows where to look for it. Backing up PATH first meant we could have restored it immediately if that had happened. It did not happen this time, but the check was worth doing before trusting the install.

4. GIT CREDENTIAL / PUSH ISSUES

Step	Result
GitHub Desktop prompted for a credential helper on first push	INFO
Selected manager-core (Git Credential Manager) - crashed with MissingMethodException	FAIL
Root cause: broken/mismatched GCM component versions on this laptop	INFO
Switched credential.helper to wincred (native Windows Credential Store)	OK
Re-ran push from Git Bash directly - succeeded (14 objects)	OK
Attempted git push origin v0.2.0 - failed: tag did not exist locally yet	FAIL
Root cause: tag was created in a separate reference environment, never created locally	INFO
Created tag locally (git tag -a v0.2.0), pushed successfully	OK

KEY ISSUE - Git Credential Manager crash

manager-core failed with 'MissingMethodException' on AzureReposHostProvider - a broken GCM install, unrelated to the repo or token. Worked around by switching to wincred. Proper fix (not urgent) would be a clean GCM reinstall.

In plain terms: A 'credential helper' is the piece of software that remembers your GitHub login so you are not asked for it on every single push. This laptop's default helper (manager-core / Git Credential Manager) had a broken internal component and crashed outright. The fix was not to repair it, but to tell Git to use a different, simpler helper (wincred, which is Windows' own built-in credential store) instead. Both achieve the same goal - remembering login details securely - just via different underlying tools.

5. REPO STATE / GIT HYGIENE ISSUES

Step	Result
Noticed prompt showing detached HEAD state (double parentheses)	FAIL
git diff on pki/scripts/issue.sh showed only a file-mode change (755 -> 644)	INFO
Root cause: NTFS does not preserve Unix executable bit reliably on copy	INFO
Set git config core.fileMode false to stop future false-positive mode diffs	OK
Discarded the mode-only change, confirmed working tree clean	OK
git checkout main - returned to branch head, detached state resolved	OK
Confirmed commit history intact (89487f2, a2e5b86, e1b9692)	OK

LESSON - Always diff before discarding

Every unexpected local change was inspected with `git diff` before being discarded, confirming each was a Windows-specific artifact (line endings / file mode) and not real content loss, before running any destructive `git` command.

In plain terms: 'Detached HEAD' means your working folder is looking at one specific past commit instead of following the tip of a branch (like `main`) as it moves forward. It usually happens by checking out a specific commit or tag directly. It is not dangerous by itself, but any new commits made while detached can become orphaned - invisible from `main` - the moment you switch back. The fix is simply '`git checkout main`' to reattach. Separately, the file-mode warning (755 to 644) was Windows/NTFS not preserving the Linux 'executable' permission bit when a file is copied - not an actual code change. '`git diff`' was used first, every time, to tell the difference between 'nothing to worry about' and 'something that would actually be lost' before deciding what to do.

6. BUILDENV.SH - STRUCTURED BUILD WRAPPER

Added `scripts/buildenv.sh`: generates dedicated per-stage artifact folders (preprocessed/asm/obj/elf/bin/map/logs), writes a sequenced timestamped log per run, and fails immediately with a loud banner (terminal + log) on the first broken build step.

Step	Result
Verified in sandbox: clean build, both ZCU targets	OK
Verified in sandbox: deliberately broken source triggers failure banner correctly	OK
Committed and tagged v0.2.0, pushed to GitHub	OK
Re-verified on local laptop: <code>arm-none-eabi-gcc --version</code> OK (14.2.Rel1)	OK
<code>./scripts/buildenv.sh</code> all clean run on laptop - both targets BUILD SUCCEEDED	OK

RESULT

Both `zcu1-discovery` and `zcu2-nucleo` compiled and linked cleanly on the actual laptop toolchain, matching sandbox results exactly (text=200, data=0, bss=2056 for both). Firmware build pipeline is now proven end-to-end on real developer hardware.

7. OTHER ADDITIONS

- Added Author / Version / Last-updated metadata block to all README.md files in the repo.
- Updated LICENSE copyright line to Chittaranjan Baral.
- Added CHANGELOG.md, tracking v0.1.0 and v0.2.0.
- Added `daily-logs/` folder (this document's home) with naming convention YYYY-MM-DD_daily-log.pdf and a README describing suggested content per entry.

8. OPEN ITEMS FOR NEXT SESSION

- STM32 Discovery/Nucleo exact part numbers not yet confirmed (Ethernet PHY question still open).
- Generic linker script (STM32_GENERIC.ld) and placeholder vector table not yet matched to real part-number datasheet values - must fix before flashing real hardware.
- RPi5 Stage 1 (stock Raspberry Pi OS flash + validation) not yet started.
- QNX SDP release with confirmed RPi5 BSP support not yet identified.
- Optional: proper GCM reinstall to fix manager-core crash (not urgent, wincred works).

9. GLOSSARY (FOR NEW READERS)

PATH	The list of folders Windows/Linux searches to find a program when you type its name without a full file path. If a folder isn't in PATH, typing that program's name gives 'command not found', even if it's installed.
Git Bash	A terminal program that lets Windows run Linux-style bash commands and scripts. Different from Windows' own cmd or PowerShell - some scripts (like buildenv.sh) only run in Git Bash, not cmd.
Credential helper	The tool Git uses to remember your GitHub login so you aren't asked for it on every push. Windows offers several (wincred, manager-core, store, cache) - they all do the same job differently.
Personal Access Token (PAT)	A long random password-like string GitHub issues instead of your real account password, used for git operations from the command line. GitHub no longer accepts your normal login password for this.
Detached HEAD	A state where your local repo is looking at one fixed past commit instead of following a branch (like main) forward. Fixed with 'git checkout main'.
Git tag	A permanent label on one specific commit, usually marking a release (e.g. v0.1.0). Unlike branches, tags don't move.
File mode / executable bit	A Linux-style permission flag marking a file as runnable. Windows/NTFS doesn't track this the same way, so copying files across the two systems can make Git think a file 'changed' when only this flag flipped.
Toolchain	The full set of programs (compiler, assembler, linker) needed to turn source code into a program the target chip can actually run. 'arm-none-eabi-gcc' is the compiler at the center of this toolchain.
mTLS (mutual TLS)	A security handshake where BOTH sides of a connection prove their identity with certificates, not just the server proving itself to the client (which is how most websites work).